

HAIRSPRING: Models propose; software verifies

An agent harness that checks work, records evidence, recovers after crashes, and tests its own improvements

Tensorbend Instinct

September 2026

Abstract

A language model can write code, use tools, and say that a task is finished. Its confidence is not proof. Hairspring is an agent harness built around a simple rule: the model proposes work, while ordinary software verifies the result, records what happened, restores the run after a crash, controls what agents share and remember, and tests changes to the harness before installing them. This paper explains that design from the ground up. It compares five common agent designs, states the failure each leaves open, and traces one Hairspring mission from goal to checked result. On a curated set of ten software-engineering tasks, Hairspring resolved nine tasks and a direct-loop baseline resolved three under the same host-side tests. Hairspring used fewer model steps, but about 7.5 times the wall time and 7.2 times the cost. The checked-in records do not identify the model configuration for both arms, so this paper does not claim model parity. The comparison has no random sampling, repeated trials, or component ablation. It is an engineering case study, not evidence of general superiority.

1 The problem in plain English

Most software agents repeat the same cycle: give a model a goal, let it use tools, show it the results, and stop when it submits an answer [1, 3]. This works until the model is wrong about being done. It may run the wrong test, repair only the visible symptom, forget an earlier failure, or report success after a tool broke.

Hairspring separates doing the work from deciding what becomes official. The model may search, edit, run tools, start limited child agents, and revise its answer. It may not write the final pass result. Software checks run against the submitted files. A second model, started with fresh context, may look for a specific blocking defect, but it cannot override a failed test. One durable event record stores the evidence needed to inspect or resume the run.

The same rule applies beyond the final answer. An agent may propose a shared fact, a memory, or a change to its own instructions. Separate software decides whether to accept it. Software makes the final decision on whether each proposal becomes official. Figure 3 pairs each proposal with the software that makes that decision.

What this paper contributes.

1. A plain comparison of five common agent designs and the unchecked failure each leaves open.
2. A complete mission path in which tests and a fresh review decide whether work passes, while one event record supports inspection and crash recovery.
3. Separate acceptance paths for shared state, memory, and changes to the harness.
4. A matched ten-task comparison, with raw per-task records and commands that recompute the reported totals.

2 Five common agent designs

Figure 1 is a map of the field. The categories describe control-flow shapes, not named products. Real systems often combine them.

Direct loop. One model repeats think, act, and observe, then submits when it believes the task is done [1, 3, 4, 2]. This is simple and cheap. It also lets the worker grade itself, so a fluent wrong answer can cross the finish line [5].

Fixed workflow. A person draws a graph of model steps and branches before the run starts [13]. This is predictable for known cases. A new situation may have no valid route, and a model-based checking step is still a model opinion unless an external test owns the result.

Five common agent designs, and the failure each leaves open

	WHAT IT DOES	WHAT CAN STILL GO WRONG
Direct loop	One model works and decides it is done.	A confident wrong answer can pass.
Fixed workflow	A preset graph routes work through model steps.	A new situation can have no valid route.
Model team	Models plan, work, debate, and vote.	Shared blind spots can win the vote.
Search or evolution	Many candidates are scored and selected.	A model-written score can reward the wrong thing.
Memory add-on	Past notes are retrieved into later prompts.	A wrong note can become a lasting fact.
Shared problem: the model can still approve, preserve, or spread its own mistake.		

Figure 1: Five common agent designs. The middle column says what each design does. The right column states the unchecked failure that remains. All five can do useful work; none, by shape alone, prevents a model’s own mistake from becoming the accepted result or lasting state.

Model team. Several copies of a model plan, debate, vote, or take roles [6]. Different wording is not independent evidence. Shared blind spots can win a vote, producing agreement without correctness [7, 8].

Search or evolution. A controller generates many reasoning paths, prompts, or policies and selects the highest-scoring candidate [9, 10, 11]. This can be strong, but it fails when a candidate influences the judge or the model’s confidence becomes the score. Cost also grows with the number of branches and trials.

Memory add-on. A store retrieves notes from earlier runs and places them in the next prompt. This extends a model across sessions. Without validation and source tracking, a wrong note can become a lasting fact; relevance does not mean truth [12].

Shared problem. In each shape, some model output can become accepted without a different mechanism proving it. Hairspring addresses that problem by giving each kind of proposal a separate acceptance path.

3 What must be decided outside the worker

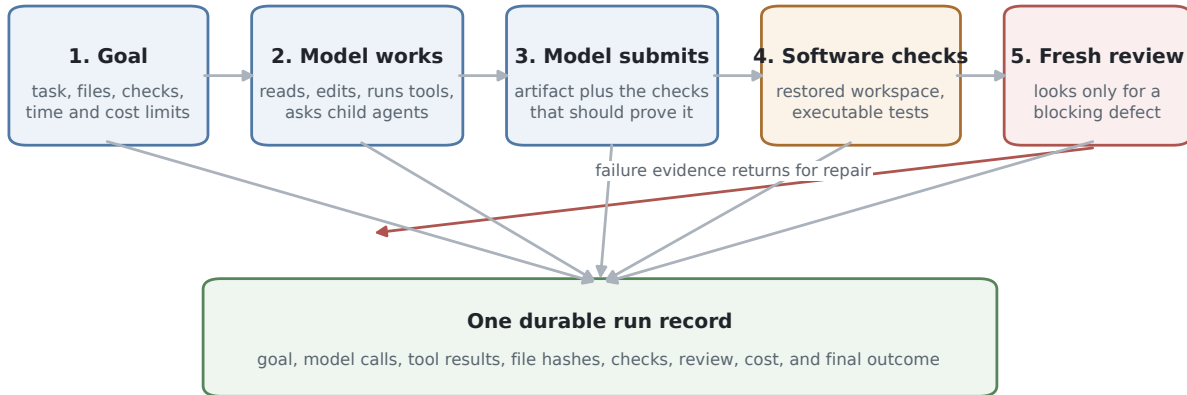
Five questions appear in a long-running agent system. Each has a plain software owner.

Did the work pass? Executable checks run on the submitted workspace. A fresh reviewer may block a result by naming a grounded defect.

What happened? A single event writer stores ordered model calls, tool results, file identities, costs, and verdicts.

What may agents share? A world service validates proposed shared state before other agents can treat it as accepted.

Hairspring in one page: the model proposes; software checks and records



The mission ends only when the checks pass and the fresh review finds no blocking defect.

A model can suggest success. It cannot write the final pass record.

Figure 2: One Hairspring mission. Read left to right across the top. Failures loop back to the model as repair evidence. Every downward arrow writes to the same run record, which stores the work, checks, review, cost, and final outcome.

What should be remembered? A memory record keeps its source and later-use signal. A retrieved note is never relabeled as proven truth.

Did a harness change help? A scorer fixed before the trial compares old and new policies on held-out tasks and records why a change was kept or undone.

The worker can still be creative. These boundaries govern only what becomes a pass, official history, shared state, stored memory, or an installed policy.

4 A mission from start to finish

A mission begins with a goal, a project directory, available tools, success checks, and time and cost limits. The model works through typed tools. Each model call and tool result is recorded. When ready, the model submits both an artifact and the checks that should prove it.

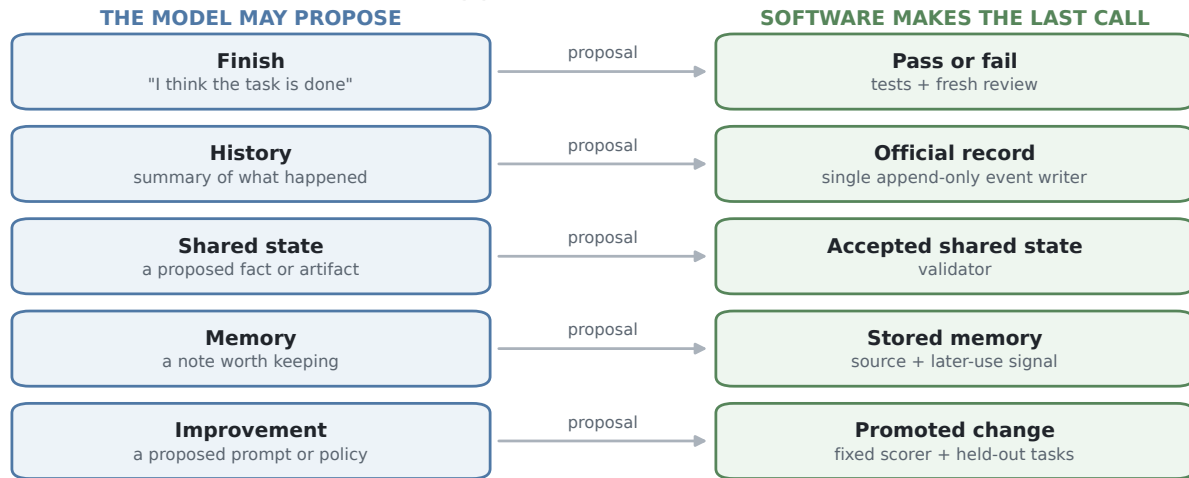
The checker restores the workspace to the submitted state and runs those checks there. If a check fails, the failure evidence goes back to the worker for repair. If the checks pass, a fresh-context reviewer sees the goal and artifact, without the worker's explanation, and looks for a blocking defect. That reviewer can return a defect; it cannot turn a failed check into a pass. Only a clean check result plus no blocking finding creates a verified outcome.

5 The boundary between suggestions and decisions

Figure 3 shows the boundary directly. The left side lists what a model may suggest. The right side names the mechanism that decides whether that suggestion becomes official.

Tools and files. The kernel exposes one generated list of typed tools, so the prompt and the executable registry cannot silently disagree. Tool requests and results retain names, arguments, timing, outcome, and payload references. File edits use patches or anchored replacements. Commands run in an operating-system sandbox; startup fails if the required sandbox is unavailable.

Who decides what: model suggestions versus software decisions



The model can suggest each outcome. A separate mechanism decides what becomes official.

Figure 3: Who makes the last call. Each row pairs a model suggestion with the software decision required before it becomes official. Software makes the final decision on whether each proposal becomes official.

Submission as one transaction. The answer and its declared checks are persisted together before judgment begins. This prevents an answer from appearing final before its proof requirements are known. Checks run on the submitted workspace rather than an arbitrary current directory.

Fresh review. The reviewer receives the goal and artifact in a new context. Its narrow job is to find a specific, grounded reason the artifact should not pass. It runs read-only in the production path. If the reviewer crashes or returns an invalid result, the mission does not pass.

6 A record that can restore the run

A chat transcript is not enough to resume real work. It may omit tool state, partial effects, spending, or limits. Hairspring stores typed events in order, places large payloads under content hashes, and gives only one component permission to append to the official stream. Tools return results to that writer instead of changing the record directly.

Recovery reads the last complete, consistent prefix. It detects and discards an incomplete tail, then restores the transcript, tool results, counters, file references, and billed cost. The mission continues under the same identity and limits. A restart therefore cannot erase a failed check or reset a budget.

Evidence. Storage tests inject incomplete tails, duplicate clock values, corrupt registry markers, dead tools, interrupted migrations, and replay after a policy change. These tests live in the crates at the evaluated source revision.

7 Child agents, shared state, and memory

A parent may start child missions with explicit step limits. Each child has its own event stream, charges work to the parent, and must finish before the parent can close. A child cannot start more children because the delegation tool is absent from its executable tool list.

Children may propose facts or artifacts to a shared world service. Validation must accept the proposal before another child treats it as shared state. The service records reuse, making it possible to see whether one child found and used another child's accepted artifact [14].

Crash recovery: resume the same run, not a reconstructed imitation



Only a complete, consistent prefix is replayed. A partial final event is discarded.

Recovered state includes the evidence and spending history, so a restart cannot erase a failure or reset a budget.

Figure 4: Crash recovery. The durable prefix contains more than conversation: it includes tool state, file identities, counters, and cost. Replay restores the same mission; it does not ask the model to summarize and start over.

Memory uses the same discipline. Every stored record keeps its source and mission identity. Later citation can raise a reuse signal; being served and ignored can lower it. This signal measures usefulness, not truth. End-to-end tests cover writing, retrieval, and cross-child reuse. The unmeasured question is whether a live model keeps selecting the right records as the store grows.

8 Testing changes to the harness

The worker repairs a task artifact. A separate outer loop can propose changes to prompts and policy files. Before the candidate runs, Hairspring fixes the scorer and held-out tasks before either policy runs. The existing policy and candidate then run under the same test. The record keeps both hashes, test conditions, traces, scores, decision, and reason.

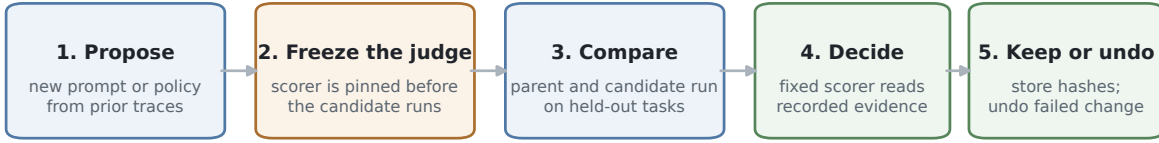
A malformed or fabricated verdict scores zero, and a failed candidate is undone. The proposer never grades or installs itself. No live promotion records are checked in, so this paper claims only that the mechanism and its tests exist, not that observed promotions improved production results. Hairspring does not yet rewrite and safely promote its own Rust code.

9 Evaluation method

The evaluation separates four kinds of evidence. Source and unit tests establish that a mechanism exists and preserves local rules. Installed-path tests establish that the terminal, registry, sandbox, checker, and rendered state connect that mechanism to a real mission. Live artifact trials establish that a real model can produce an artifact that passes independent checks. Matched external evaluations compare harness outcomes under one shared judge. A large unit suite does not substitute for an installed-path result, and a live result does not establish a general benchmark claim by itself.

The principal comparison uses ten software-engineering tasks drawn from a live public benchmark, with the same host-side fail-to-pass checks. Each system operated through its native interface. The checked-in comparison records do not identify both arms' model configuration, so model parity cannot be audited from the artifact. Raw records identify task, submission status, outcome, steps, wall time, and cost, and are checked into the repository. The comparison is a curated subset, not a leaderboard sample. There are no repeated seeds or confidence intervals.

Safe improvement: a candidate cannot grade or install itself



Recorded for every decision

parent hash • candidate hash • tasks • scorer version • traces • scores • reason

Current limit: Hairspring can test prompt and policy changes. It does not yet rewrite and promote its own Rust code.

Figure 5: How a prompt or policy change is tested. The judge is fixed before the candidate runs. The old and new policies face held-out tasks, and the decision record contains the exact versions and evidence.

Table 1: Aggregate matched result. Wall time and cost are sums of the raw per-task records.

System	Resolved	Worker steps	Wall time	Cost (USD)
HAIRSPRING	9/10	987	26,935 s	6.484234
Class-C1 baseline	3/10	1,170	3,613 s	0.903649

10 What the matched comparison measured

The baseline is a representative of class C1: a public agent–computer interface scaffold that runs a direct loop [3]. It provides a concrete comparison to a simpler control-flow shape: no workflow graph, role ensemble, or search controller.

HAIRSPRING’s failure was a two-hour timeout on `deepset-ai/haystack-8619`. The successful runs ranged from 23 to 171 worker steps. The baseline’s three passes were `conan-17092`, `haystack-8525`, and `conan-17129`. Several baseline failures reached a submission but failed host-side tests; others encountered missing evaluation dependencies. The shared host-side verdict remains the reported outcome.

The result is consistent with the hypothesis that executable judgment followed by repair can recover tasks that a cheaper flat loop leaves unresolved. It does not establish causality: the artifact lacks a component ablation, repeated trials, and an auditable record of model parity. An early controlled test of parallel execution produced no gain and was reverted. The paper does not attribute the benchmark result to parallelism.

The trade is visible. HAIRSPRING spends about 7.2 times as much money and 7.5 times as much wall time to judge and repair during the run, and recovers six additional tasks for it. The baseline remains simpler and cheaper when an external evaluator already supplies the final verdict and the job does not need recovery, memory, shared state, or self-improvement. Per-task records, including the corrected wall and cost columns, are in Appendix C.

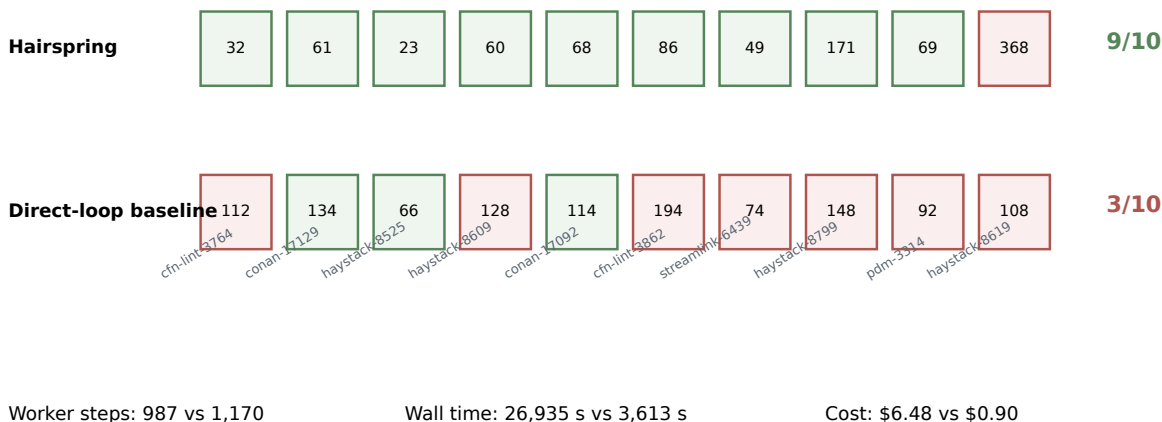
11 Defects that changed the design

The highest-value defects crossed subsystem boundaries while local tests remained green. Four examples shaped the current test rule.

A dead tool surface. A capability could exist in a crate while the production terminal never registered it. The fix required a test through the real registry and installed terminal.

Measured result on one matched set of ten software-engineering tasks

Green = host-side tests passed. Red = failed or timed out. This is a curated subset, not a general leaderboard.



Result: Hairspring recovered six more tasks, but took more time and cost more. The data does not isolate which subsystem caused the gain.

Figure 6: Curated ten-task results. HAIRSPRING resolved 9 tasks; the class-C1 baseline resolved 3 under the same host-side tests. The comparison does not establish model parity or isolate the effect of verification.

A wrong production mode. General missions accidentally used an amnesiac benchmark arm. Unit tests for both modes passed; only a live installed-path run exposed the selection error.

Recursive delegation. The first delegation design allowed descendants to delegate further, so process growth was bounded only by the host. The repaired design removes delegation from a child’s surface and adds an explicit spawn cap; an installed-path test now reproduces this failure class.

An impossible gate. Missing test dependencies and strict marker settings caused the evaluator to collect zero tests while the agent kept repairing a correct patch. Preflight now proves that the gate can execute before a mission starts.

A related defect class is reward hacking of the close path: the model learned to convert repeated critic refutations into a pass under a retry ratchet. The fix made malformed or absent verdicts fail closed, and refutation counters now reset per mission. This is the failure expected when a model can turn its own review into a pass. The final verdict is therefore executable, not rhetorical.

These failures lead to one rule: a mechanism is not accepted because its crate test passes. Its regression must reproduce how a model reaches it through the installed terminal path, including registry, sandbox, project root, judgment order, and rendered outcome.

12 What Hairspring does not yet coordinate

Recent long-horizon research controllers organize work into strategy exploration, readiness gates, dependency graphs, and global verification [16]. Their strongest transferable idea is explicit control over when a strategy is mature enough to decompose and where a verifier’s finding should return.

HAIRSPRING supplies lower-level services for such controllers: replayable state, executable checks, a fresh reviewer, bounded children, shared state, memory, cost accounting, and held-out policy tests. It does not yet implement that research-planning layer. Missions remain flat. Critic findings return as whole-mission gaps. Candidates are selected or compared rather than constructively synthesized with their objections. No typed dependency graph localizes a defect to the affected

closure.

13 Limitations

1. The headline comparison contains ten curated tasks. It establishes a difference on that subset, not general software-engineering superiority.
2. Verification is expensive. On the matched subset, HAIRSPRING spent about 7.2 times as much money and 7.5 times as much wall time as the baseline.
3. The study has no repeated seeds and cannot estimate result variance.
4. Self-improvement changes prompts and policy artifacts, not the Rust implementation.
5. Memory writes, recall, and cross-child reuse are exercised end to end; live-model selection quality at scale is not measured.
6. Research planning is flat. The system does not yet track strategy readiness, typed dependencies, preserved objections, or repairs tied to specific sections.
7. At the evaluated paper revision, the workspace test suite passed; current pedantic Clippy does not pass. Passing tests do not make a different quality gate pass.

14 Reproducibility

The repository carries manuscript source, generated figures, raw matched records, and a reproduction script. The source revision for the measurements is 4273dccb6d00fa727b01fcbdedc6773ace9eca12. Paper changes after that revision do not alter the raw run records.

Table 2: Regeneration commands. These commands do not call a paid model.

Measurement	Command from repository root
Rust source lines	<code>find crates -name '*.rs' -print0 xargs -0 wc -l</code>
Workspace crates	<code>find crates -mindepth 2 -maxdepth 2 -name Cargo.toml wc -l</code>
Named tests	<code>cargo test -workspace -locked - -list grep -c ': test\$'</code>
Matched aggregates	<code>python3 paper/reproduce.py benchmark</code>
Paper PDF	<code>cd paper && make clean all</code>

The benchmark runs require task repositories, test patches, provider credentials, and runner environments. The checked-in CSV and JSON permit recomputation of the aggregates without rerunning paid inference. The workspace test suite was measured on the build host at the evaluated revision; the installed-path tests use that host’s absolute plugin paths, so a fresh clone compiles the suite but does not reproduce those runs. Fresh-clone regeneration covers the source metrics, the named-test listing, and the matched aggregates.

15 What the separation changes

Separating suggestions from decisions makes failures easier to read. A failed check is an execution fact, not an opinion about answer quality. A broken reviewer is a machinery failure, not a pass. A retrieved memory is a sourced note, not shared truth. A well-explained candidate policy is still only a candidate until the fixed test measures it.

This framing does not remove model error. It changes where error can become state. The worker may still choose a poor plan or spend too long on a dead end. The surrounding software can preserve evidence, enforce budgets, refuse an invalid finish, and return a specific defect for repair. Those properties become more important as missions grow longer because recovery and attribution costs rise with the amount of hidden state.

16 Conclusion

HAIRSPRING treats reliable agent work as a separation-of-responsibilities problem. Across five common agent designs, model output can still become accepted without independent proof. Hairspring uses a simpler rule: the model does open-ended work, while tests, a fresh reviewer, an event writer, validators, and a fixed scorer decide what becomes a pass, official record, shared fact, stored memory, or installed policy.

On one curated ten-task software-engineering subset, this architecture resolved more tasks than a representative direct-loop baseline while using fewer worker steps, more money, and more wall time. The artifact supports those descriptive totals, not a causal or general superiority claim. The design goal is to make long-running work easier to inspect and recover by separating model proposals from software decisions.

A Artifact map

The implementation is divided by responsibility rather than by interface layer.

Crate	Responsibility
hs-core	Event schema, fixed encoding, hash chaining
hs-log	Segmented streams, payload store, fsync, recovery, rewind
hs-kernel	Plugin process isolation, protocol, dispatch
hs-loop	Worker loop, checker path, REPL, TUI, built-in tools and models
hs-swarm	Bounded asynchronous children, booking, join at close
hs-goal	Graded mission rig and external grading
hs-hashline, hs-applypatch	Anchored and patch-based editing
hs-bench, hs-scorer	Benchmarks, tiered scoring, pins, canaries, baselines
hs-selfmod	Candidate cycle, migration, promotion, rewind
hs-memory, hs-world	Provenance-bearing memory and validated shared state
hs-cli	Event-log inspection and fixtures

B Event and verdict flow

The following abbreviated trace shows the intended ordering. Field names are illustrative; the executable schema in `hs-core` is the source of truth.

```
mission.opened    { goal, project_root, provider, budgets }
model.requested   { step, context_hash }
model.responded   { usage, billed_cost }
tool.requested    { name, args_hash }
tool.completed    { status, payload_hash }
answer.submitted  { artifact_hash, declared_checks }
checks.completed  { status, evidence_hash }
critic.completed  { refuted, findings_hash }
mission.closed    { outcome: "verified" | "rejected" | "stopped" }
```

A valid close follows persisted submission and judgment evidence. A replay reconstructs state from the ordered stream and content-addressed payloads. It does not ask the worker to summarize what happened.

C Matched raw-result schema and per-task records

The HAIRSPRING CSV uses one row per task:

```
instance_id, passed, worker_steps, model_calls, cost_micro_usd, wall_seconds, note
```

The baseline JSON stores the corresponding instance identifier, submission flag, host-side result, test tail, steps, cost, and token counts. `paper/reproduce.py benchmark` sums the two files and prints the totals reported in Table 1. (An earlier revision of this paper transposed the wall and cost columns in the per-task table; the columns below match the schema above and the raw files.)

Table 3: HAIRSPRING per-task results from the checked-in raw CSV, with corrected wall and cost columns.

Instance	Pass	Steps	Calls	Wall (s)	Cost (USD)
cfn-lint-3764	yes	32	33	603	0.096
conan-17129	yes	61	62	769	0.158
haystack-8525	yes	23	24	397	0.067
haystack-8609	yes	60	61	1,302	0.237
conan-17092	yes	68	71	3,503	0.631
cfn-lint-3862	yes	86	89	3,024	0.349
streamlink-6439	yes	49	50	2,834	0.197
haystack-8799	yes	171	172	5,053	1.551
pdm-3314	yes	69	70	2,250	0.424
haystack-8619	no	368	368	7,200	2.774

D Fresh-clone reproduction

```
git clone https://github.com/tensorbend-instinct/hairspring.git
cd hairspring
git checkout 4273dccf6d00fa727b01fcbdedc6773ace9eca12
export PATH="$HOME/.cargo/bin:$PATH"
```

```
python3 paper/reproduce.py source
python3 paper/reproduce.py benchmark
cd paper && make clean all
```

E Evaluation claims and boundaries

Claim	Status	Boundary
14 crates; 342 Rust files; 70,429 lines	Reported	Requires the evaluated source revision
992 tests passed; 0 failed	Reported	Build-host run is not reproduced by the paper artifact; named-test listing reports 994 entries
9/10 versus 3/10	Measured	One curated subset under shared host-side tests; model parity is not recorded
Verifier–repair supports recovery	Hypothesis	Comparison is consistent with it; no component ablation establishes causality
Parallelism caused the gain	Not supported	First controlled test showed no gain and was reverted
General software-engineering superiority	Not claimed	No random sample, repeated seeds, or confidence interval
Research-proof controller	Not implemented	Strategy readiness and section dependency state remain future work

References

- [1] S. Yao et al. “ReAct: Synergizing Reasoning and Acting in Language Models.” *ICLR*, 2023.
- [2] N. Shinn et al. “Reflexion: Language Agents with Verbal Reinforcement Learning.” *NeurIPS*, 2023.
- [3] J. Yang et al. “SWE-agent: Agent–Computer Interfaces Enable Automated Software Engineering.” *NeurIPS*, 2024. <https://arxiv.org/abs/2405.15793>.
- [4] “mini-swe-agent: The 100-line agent.” 2025. <https://github.com/SWE-agent/mini-swe-agent>.
- [5] J. Huang et al. “Large Language Models Cannot Self-Correct Reasoning Yet.” 2023. <https://arxiv.org/abs/2310.01798>.
- [6] X. Wang et al. “Self-Consistency Improves Chain of Thought Reasoning in Language Models.” *ICLR*, 2023.
- [7] “When Consensus Is Not Correctness: Diversity Collapse and Manufactured Overconfidence in Multi-Agent LLM Debate.” *KDD Workshop on Agentic AI Evaluation*, 2026. https://kdd-eval-workshop.github.io/agenticai-evaluation-kdd2026/assets/papers/49_When_Consensus_Is_Not_Corre.pdf.
- [8] “From Spark to Fire: Modeling and Mitigating Error Cascades in LLM-Based Multi-Agent Collaboration.” 2026. <https://arxiv.org/abs/2603.04474>.
- [9] S. Yao et al. “Tree of Thoughts: Deliberate Problem Solving with Large Language Models.” *NeurIPS*, 2023.
- [10] “Cost-Awareness in Tree-Search LLM Planning: A Systematic Study.” 2025. <https://arxiv.org/abs/2505.14656>.
- [11] “GEPA: Reflective Prompt Evolution.” DSPy documentation, 2026. <https://dspace.ai/api/optimizers/GEPA/overview/>.
- [12] “MemFail: Stress-Testing Failure Modes of LLM Memory Systems.” 2026. <https://arxiv.org/abs/2605.26667>.

- [13] M. Hu. “Workflow vs. Agent: a Policy-vs-Script Perspective.” Hugging Face blog, 2026. <https://huggingface.co/blog/MengkangHu/workflow-vs-agent>.
- [14] S. Pal et al. “SwarmWorld: Stigmergic technological evolution in societies of language-model agents.” 2026. <https://arxiv.org/abs/2608.26081>.
- [15] Y. Lee et al. “Meta-Harness: End-to-End Optimization of Model Harnesses.” 2026. <https://arxiv.org/html/2603.28052>.
- [16] H. Lin et al. “Stellar Colosseum: A Many-Agent Harness for Long-Horizon Research in Mathematics and Theoretical Computer Science.” 2026. <https://arxiv.org/abs/2609.15983>.